

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO4 – Articulation (combining methods for evaluation) P4	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL ₅)		
Student Name:	G. Tejasree	Reg. No.:	192511204
Section / Batch:	Section A	Date:	30 August 2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	

AR Motion Tracking Constraint Problem

1. Industry Problem

An **Augmented Reality (AR) system** overlays virtual objects onto the real-world environment. For a good AR experience, the virtual objects must remain stable even when the user moves the mobile device.

The system has two important constraints:

1. **Stable tracking:** The tracking drift must be **less than 5%** so that virtual objects do not move away from their correct positions.
2. **Mobile hardware:** The system must run efficiently on mobile devices, where processing power and battery capacity are limited.

The main challenge is therefore to select a motion-tracking method that provides sufficient accuracy while also maintaining real-time performance.

2. Application of Concepts

Two possible approaches are **Optical Flow** and **Motion Estimation**.

Optical Flow:

Optical Flow determines the apparent movement of pixels between consecutive video frames. It provides detailed information about how objects and features move in the scene. This can provide accurate tracking, but processing many image features requires considerable computational power.

Motion Estimation:

Motion Estimation determines the overall movement of the camera or important features between frames. It is comparatively efficient and suitable for real-time applications, but it may provide less detailed information than Optical Flow.

3. Comparison

Factor	Optical Flow	Motion Estimation
Tracking detail	High	Moderate
Accuracy	High	Moderate to high
Computational requirement	High	Lower

Mobile suitability	Moderate	High
Real-time performance	More difficult	Better
Battery usage	Higher	Lower
Suitable for continuous tracking	Yes, but expensive	Yes

4.Solution Design

For this AR system, **Motion Estimation should be used as the primary tracking method** because the system must operate on mobile hardware. Its lower computational requirement makes it more suitable for real-time tracking.

To satisfy the **less than 5% drift constraint**, the system can use a **hybrid design**. Motion Estimation can perform continuous tracking, while Optical Flow can be applied when additional accuracy is needed, such as when tracking confidence decreases or noticeable drift occurs. The basic design can be:

Camera Input → Feature Detection → Motion Estimation → Position Update → Drift Check → Optical Flow Correction if Required

This approach avoids using the computationally expensive method continuously.

4. Proposed Solution Design

A **hybrid approach** is proposed.

Working

Camera Input → Feature Detection → Motion Estimation → Position Update → Drift Check → Optical Flow Correction

1. Camera captures the current frame.
2. Important features are detected.
3. Motion Estimation calculates the movement.
4. The position of the virtual object is updated.
5. Tracking drift is checked.
6. Optical Flow is used when additional correction is required.

Motion Estimation is used continuously because it is computationally efficient. Optical Flow is used selectively when tracking confidence decreases or noticeable drift occur

5. Python Implementation Using OpenCV

```
import cv2
import numpy as np

# Open camera
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Unable to open camera")
    exit()

# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Unable to read frame")
    cap.release()
    exit()

# Convert to grayscale
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Detect important features
old_points = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

print("AR Motion Tracking Started") print("Press
Q to stop")

while True:

    ret, frame = cap.read()
    if not
ret:
break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Optical Flow    if
old_points is not None:
```

```

        new_points, status, error = cv2.calcOpticalFlowPyrLK(
old_gray,
    gray,
old_points,
    None
)

```

```

if new_points is not None:

```

```

    good_new = new_points[status == 1]
    good_old = old_points[status == 1]

```

```

    if len(good_new) > 0:

```

```

        # Calculate motion

```

```

dx = np.mean(
    good_new[:, 0] - good_old[:, 0]
)

```

```

dy = np.mean(
    good_new[:, 1] - good_old[:, 1]
)

```

```

    movement = np.mean(
np.linalg.norm(
    good_new - good_old,
        axis=1
    )
)

```

```

        # Draw motion vectors
    for
new, old in zip(good_new, good_old):

```

```

        x1, y1 = new.ravel()
        x2, y2 = old.ravel()

```

```

        cv2.arrowedLine(
frame,
            (int(x2),
int(y2)),
            (int(x1), int(y1)),
            (0, 255, 0),
            1
        )

```

```

        cv2.circle(
frame,
            (int(x1), int(y1)),

```

```

        3,
        (0, 0, 255),
        -1
    )

    # Display values
    cv2.putText(
    frame,
        f'Motion X: {dx:.2f}',
        (20, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    cv2.putText(
    frame,
        f'Motion Y: {dy:.2f}',
        (20, 60),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    cv2.putText(
        frame,
        f'Optical
Flow: {movement:.2f}',
        (20, 90),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    # Tracking status
    if movement < 2:
        status_text = "Stable Tracking"
    elif movement < 10:
        status_text = "Moderate Motion"
    else:
        status_text = "High Motion"

    cv2.putText(
    frame,
    status_text,
        (20, 125),
        cv2.FONT_HERSHEY_SIMPLEX,

```



```

        ...,
        (255, 255, 255),
        2
    )

    # Update feature points
    old_points = good_new.reshape(-1, 1, 2)

    old_gray = gray.copy()

    # Display output
    cv2.imshow("AR Motion Tracking", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release() cv2.destroyAllWindows()

```

7. Pseudocode

START

Open camera

Capture first frame

Convert frame to grayscale

Detect important features

WHILE camera is running:

 Capture new frame

 Convert to grayscale

 Calculate Optical Flow

 Calculate feature movement

 Calculate X and Y motion

 Perform Motion Estimation

 IF tracking is stable:

 Continue Motion Estimation

 ELSE:

 Apply Optical Flow correction

 Display tracking information

Update previous frame

IF Q is pressed:
STOP

END

6. Algorithm

Step 1: Start the camera.

Step 2: Capture the first video frame.

Step 3: Convert the frame to grayscale.

Step 4: Detect important features using feature detection.

Step 5: Capture the next frame.

Step 6: Apply Optical Flow to track feature movement.

Step 7: Calculate X and Y motion.

Step 8: Perform motion estimation.

Step 9: Check tracking stability.

Step 10: Use Optical Flow correction when additional accuracy is required.

Step 11: Continue until the user presses Q.

Step 12: Stop the camera.



8. Use of Tools

The system can use:

- OpenCV for motion tracking and optical-flow processing.
- Mobile camera sensors for capturing frames.
- Accelerometer and gyroscope data to improve motion estimation.

9. Analysis and Decision

Optical Flow provides more detailed motion information, but its computational cost can make it unsuitable for continuous use on mobile hardware. Motion Estimation is faster and consumes fewer computational resources, making it better for real-time mobile AR.

Since the problem requires **both stable tracking (<5% drift) and mobile efficiency**, using only Motion Estimation may not always provide sufficient accuracy, while using only Optical Flow may reduce mobile performance.

Therefore, a **hybrid Motion Estimation + selective Optical Flow approach** provides the best balance between the two constraints.

10. Conclusion

The recommended solution is to use **Motion Estimation as the main tracking technique and Optical Flow for accuracy correction when required**. Therefore, a **hybrid Motion Estimation + selective Optical Flow approach** provides the best balance between the two constraints

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	